

1. Bevezetés

A projekt egy modern, nyílt forráskódú terminálalkalmazás, amely a klasszikus parancssoros környezetet ötvözi grafikus és mesterséges intelligenciával támogatott funkciókkal. A rendszer célja, hogy a fejlesztők, rendszergazdák és haladó felhasználók számára egy egységes munkakörnyezetet biztosítson, ahol a terminál használata gyorsabb, átláthatóbb és produktívabb lehet.

A projekt alapját a Wave Terminal adja, amely több platformon működő, modern terminálmegoldásként ismert. A rendszer különlegessége, hogy nem csupán hagyományos parancssori felületet biztosít, hanem képes fájlok, képek, markdown dokumentumok és egyéb tartalmak közvetlen megjelenítésére is.

A projekt fejlesztése során a hangsúly a következő szempontokra került:

- modern felhasználói élmény,
- gyors és hatékony terminálhasználat,
- AI-integráció,
- többplatformos működés,
- fejlesztőbarát kialakítás,
- munkafolyamatok egyszerűsítése.

2. A projekt célja

A rendszer célja egy olyan terminálalkalmazás létrehozása, amely túlmutat a hagyományos CLI (Command Line Interface) működésen. A klasszikus terminálok egyik legnagyobb problémája, hogy a felhasználónak folyamatosan különböző alkalmazások között kell váltania:

- terminál,
- böngésző,
- dokumentáció,
- szövegszerkesztő,
- fájlkezelő,
- AI-asszisztens.

A Wave Terminal ezt a problémát úgy oldja meg, hogy ezeket az eszközöket egyetlen környezetbe integrálja.

A projekt elsődleges céljai:

1. A fejlesztői munkafolyamatok gyorsítása.
2. A terminálhasználat modernizálása.
3. Kontextusváltások csökkentése.
4. AI-alapú segítség biztosítása.
5. Távoli szerverek kényelmes kezelése.
6. Grafikus elemek integrálása terminálkörnyezetbe.

3. A rendszer fő funkciói

3.1 Terminál emuláció

A rendszer alapfunkciója egy fejlett terminál emulátor biztosítása. A felhasználó ugyanúgy futtathat shell parancsokat, mint hagyományos Linux vagy Windows terminálban.

Támogatott műveletek:

- shell parancsok futtatása,
- script kezelés,
- fájlműveletek,
- Git használat,
- SSH kapcsolatok,
- rendszeradminisztrációs feladatok.

3.2 AI integráció

A rendszer egyik legfontosabb eleme a mesterséges intelligencia integrációja. Az AI képes:

- terminálkimenetek elemzésére,
- hibák felismerésére,
- parancsok ajánlására,
- fájlmodosításokra,
- fejlesztői segítség nyújtására.

A rendszer támogatja több AI szolgáltató használatát is:

- OpenAI
- Anthropic
- Google
- Ollama

A felhasználó saját API kulcsot is használhat.

3.3 Inline renderelés

A rendszer képes különböző fájlok közvetlen megjelenítésére terminálon belül.

Támogatott tartalmak:

- képek,
- markdown fájlok,
- PDF-ek,
- videók,
- CSV fájlok,
- könyvtárstruktúrák.

Ez jelentősen csökkenti az alkalmazások közötti váltás szükségességét.

3.4 Távoli kapcsolatok kezelése

A rendszer fejlett SSH támogatással rendelkezik.

Funkciók:

- automatikus újracatlakozás,
- kapcsolatmegtartás hálózati hiba esetén,
- távoli fájl szerkesztés,
- távoli fájl másolás,
- több szerver egyidejű kezelése.

A rendszer egyik legfontosabb tulajdonsága, hogy az SSH munkamenetek újraindítás után is megmaradhatnak.

3.5 Workspace rendszer

A rendszer támogatja a munkaterületek használatát.

A workspace-ek lehetővé teszik:

- több terminál egyidejű használatát,
- projektek elkülönítését,
- különböző nézetek mentését,
- fejlesztői környezetek gyors visszaállítását.

4. Technológiai háttér

4.1 Programozási nyelvek

A projekt több technológia kombinációját használja.

Főbb nyelvek:

Nyelv	Szerep
Go	Backend és rendszerfolyamatok
TypeScript	Frontend logika
JavaScript	Webes funkcionalitás
CSS / LESS	Felhasználói felület
Shell	Scriptelés

A projekt főként Go és TypeScript alapokra épül.

4.2 Electron keretrendszer

Az alkalmazás az Electron technológiára épül.

Az Electron lehetővé teszi:

- cross-platform működést,
- webes UI használatát,
- natív asztali alkalmazás létrehozását,
- modern grafikus megjelenést.

4.3 React alapú felület

A frontend komponensek React technológiával készültek.

Előnyök:

- komponensalapú felépítés,
- gyors UI frissítés,
- modern fejlesztési modell,
- könnyű bővíthetőség.

5. Rendszerarchitektúra

5.1 Magas szintű architektúra

A rendszer három fő rétegből áll:

Frontend réteg

Feladata:

- grafikus felület megjelenítése,
- eseménykezelés,
- felhasználói interakciók.

Backend réteg

Feladata:

- terminálfolyamatok kezelése,
- SSH kapcsolatok,
- AI kommunikáció,
- fájlműveletek.

Rendszerszintű réteg

Feladata:

- operációs rendszerrel való kommunikáció,
- shell folyamatok kezelése,
- hálózati kapcsolatok.

6. Felhasználói felület

6.1 Modern dizájn

A rendszer modern, minimalista megjelenést használ.

Jellemzők:

- sötét mód támogatás,
- drag-and-drop elemek,
- reszponzív panelek,
- lebegő ablakok,
- testreszabható nézetek.

6.2 Command Blocks

A rendszer külön blokkokba rendezi a parancsokat.

Előnyei:

- jobb átláthatóság,
- egyszerű hibakeresés,
- gyors visszakereshetőség,
- izolált parancskezelés.

7. Előnyök

A rendszer legfontosabb előnyei:

- modern felhasználói élmény,
- AI támogatás,
- cross-platform működés,
- távoli kapcsolatok kezelése,
- inline tartalom megjelenítés,
- workspace támogatás,
- nyílt forráskód.

8. Hátrányok

A rendszer hátrányai:

- magasabb erőforrásigény,
- Electron alapú memóriahasználat,
- összetettebb működés,
- tanulási görbe kezdők számára.

9. Biztonság

A rendszer támogatja:

- natív titkosított kulcstárolást,
- SSH hitelesítést,
- API kulcskezelést,
- lokális adatkezelést.

A fejlesztők külön hangsúlyt fektetnek arra, hogy az érzékeny adatok a felhasználó gépén maradjanak.

10. Fejlesztési lehetőségek

A projekt továbbfejleszthető az alábbi irányokban:

- jobb AI automatizálás,
- plugin rendszer,
- több platform támogatása,
- teljesítményoptimalizálás,
- fejlettebb testreszabhatóság,
- cloud szinkronizáció.

11. Fejlesztési módszertan alkalmazása a projektben

A projekt fejlesztése során több szoftverfejlesztési módszertan elemei kerültek felhasználásra. A rendszer összetettsége miatt nem kizárólag egyetlen fejlesztési modellt alkalmaztunk, hanem a vízesésmodell és az agilis fejlesztés bizonyos részeit kombináltuk.

A fejlesztés kezdeti szakaszában vízesésmodell jellegű tervezést alkalmaztunk. Ennek során először meghatároztuk a rendszer fő célját, funkcióit és architektúráját. Ebben a fázisban készült el a terminálrendszer alapfelépítése, a modulok meghatározása, valamint annak megtervezése, hogy milyen technológiák kerüljenek felhasználásra. Meghatároztuk például az Electron keretrendszer használatát, a React alapú frontend kialakítását, valamint a Go és TypeScript technológiák szerepét a projektben. A vízesésmodell elemei főként az előzetes dokumentáció és a rendszertervezés során jelentek meg, mivel ezek a részek stabil alapot biztosítottak a további fejlesztéshez.

A tényleges implementáció során azonban inkább agilis fejlesztési módszertant alkalmaztunk. A rendszer fejlesztése kisebb részekre bontva történt, ahol az egyes funkciók külön iterációkban készültek el. Először a terminál alapfunkciói kerültek kialakításra, ezt követte az SSH kapcsolatok támogatása, majd az AI integráció és az inline renderelési funkciók fejlesztése. Ez a módszer lehetővé tette, hogy a rendszer folyamatosan tesztelhető és bővíthető maradjon.

Az agilis módszertan alkalmazása különösen fontos volt az AI funkciók fejlesztésénél, mivel ezek a komponensek gyakori módosítást és finomhangolást igényeltek. A fejlesztés során minden új funkció külön tesztelésre került, majd csak ezután integráltuk a fő rendszerbe. Ez csökkentette a hibák számát és stabilabb működést biztosított.

A projekt során moduláris fejlesztési megközelítést is alkalmaztunk. A rendszert különálló komponensekre bontottuk, például:

- terminálkezelő modul,
- AI modul,
- SSH kezelő,
- renderelő rendszer,
- grafikus felhasználói felület,
- workspace kezelő rendszer.

Ez a felépítés lehetővé tette, hogy az egyes részegységek egymástól függetlenül fejleszthetők és tesztelhetők legyenek. A moduláris struktúra előnye volt továbbá, hogy egyszerűbbé vált a hibakeresés és a rendszer bővítése.

A fejlesztés során Git alapú verziókezelést használtunk. A GitHub lehetővé tette a fejlesztési ágak kezelését, az egyes módosítások nyomon követését, valamint a korábbi verziók visszaállítását. Az új funkciókat külön branchekben készítettük el, majd tesztelés után integráltuk a fő projektbe. Ez a módszer biztonságosabb és átláthatóbb fejlesztési folyamatot biztosított.

A tesztelés több szinten történt. Az alapfunkciók ellenőrzése után integrációs tesztet végeztünk annak érdekében, hogy az egyes modulok megfelelően együttműködjenek. Emellett felhasználói szintű tesztelést is alkalmaztunk, amely során valós terminálhasználati helyzetekben vizsgáltuk a rendszer működését.

Összességében elmondható, hogy a projekt fejlesztése során a vízesésmodell főként a tervezési szakaszban, míg az agilis módszertan a gyakorlati fejlesztés és tesztelés során került alkalmazásra. A két megközelítés kombinációja stabil alapot és egyben rugalmas fejlesztési folyamatot biztosított a rendszer számára.

12. Összegzés

A Wave Terminal egy modern, AI-integrált terminálrendszer, amely jelentősen kibővíti a hagyományos parancssoros környezet lehetőségeit. A projekt sikeresen ötvözi a terminálok sebességét és hatékonyságát a modern grafikus rendszerek kényelmével.

A rendszer különösen hasznos lehet:

- fejlesztők,
- DevOps szakemberek,
- rendszergazdák,
- haladó Linux felhasználók számára.

A projekt legnagyobb erőssége az AI-integráció, az inline renderelés és a tartós SSH munkamenetek támogatása. A modern architektúra és a nyílt forráskód lehetővé teszi a közösségi fejlesztést és a folyamatos fejlődést.